REPORTE DE PRÁCTICA NO. 6

ANÁLISIS SINTÁCTICO DE EXPRECIONES ARITMÉTICAS

ALUMNO:

Contreras Hernandez Diego Adonay
Martinez Delgado Baruchs Jesua
Vite Juares Alberick
Navarrete Torres Oscar

PROFESOR: Dr. Eduardo Cornejo-Velázquez

1. Introducción

Durante esta practica se detalla el diseño e implementación de un analizador lexico-sintáctico orientado al procesamiento de expresiones aritmeticas. En el ambito de la computacion la construccionde compiladores e interpretes es fundamental para permitir una comunicacion entre el leguaje humano y el lenguaje de las computadoras. Para esta practica utilzamos herramientas como Flex y Bison. El objetivo principal es desarrollar un pequeño sistema capaz de reconocer tokens numericos e identificadores y validar gramaticalmente operaciones de suma, resta, multiplicacion y division, siguiendo reglas de precedencia.

2. Marco teórico

El desarrollo de un traductor o compilador se divide en varias fases secuenciales. Esta práctica se centra en las dos primeras: el análisis léxico y el análisis sintáctico.

1. Análisis Léxico (Escaneo) Es la primera fase de un compilador.lee el flujo de caracteres de entrada y los agrupa en unidades con significado propio llamadas tokens.

Tokens: Son las categorías gramaticales del lenguaje (SUMA, ID, ENTERO).

Lexema: Es la secuencia de caracteres real que coinciden con un token (”3.1416” es el lexema para el token DECIMAL).

Autómatas Finitos Deterministas (AFD): Son modelos matemáticos que sirven para reconocer lenguajes regulares. Flex utiliza estos autómatas para identificar patrones definidos mediante expresiones regulares.

2. Análisis Sintáctico (Parsing) Esta fase recibe los tokens generados por el analizador léxico y determina si la estructura de la oración u operación es válida según las reglas gramaticales.

Gramáticas Libres de Contexto (GLC): Son un conjunto de reglas que definen cómo se pueden combinar los tokens. Se representan comúnmente mediante la notación BNF (Backus-Naur Form).

Jerarquía de Operadores: Para que una calculadora funcione correctamente, el analizador debe aplicar reglas de precedencia. En este proyecto, se implementa una estructura jerárquica donde la multiplicación y división tienen mayor prioridad que la suma y resta.

Árbol de Análisis Sintáctico: Es una representación jerárquica que muestra cómo una cadena de entrada se deriva de las reglas de la gramática.

4. Manejo de Errores Un compilador robusto debe ser capaz de identificar y reportar errores en dos niveles:
Error Léxico: Ocurre cuando el escáner encuentra un carácter que no pertenece a ningún token válido (ej. un símbolo @ en una expresión aritmética).
Error Sintáctico: Ocurre cuando los tokens son válidos pero su orden no respeta las reglas de la gramática (ej. 5 + * 3).

3. Herramientas empleadas

(Flex y Bison) Flex (Fast Lexical Analyzer): Es una herramienta para generar escáneres. Su entrada es un archivo .l que contiene expresiones regulares y su salida es código fuente en C que implementa el autómata.

Bison (Yacc compatible): Es un generador de analizadores sintácticos de propósito general. Convierte una gramática libre de contexto en un analizador LALR (Look-Ahead Left-to-Right). Utiliza una pila para procesar los tokens y realizar reducciones gramaticales.

4.Objetivos

general

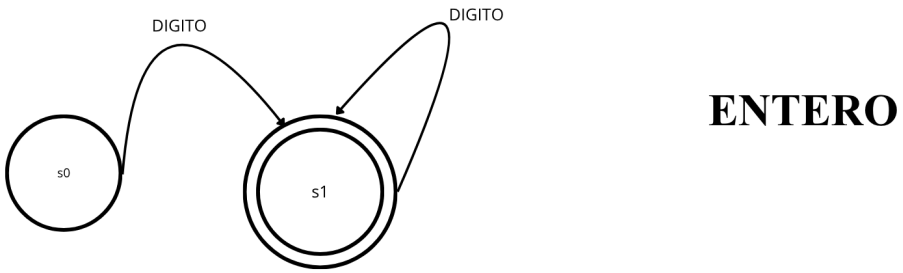
Desarrollar un analizador léxico-sintáctico capaz de validar y resolver operaciones aritméticas.

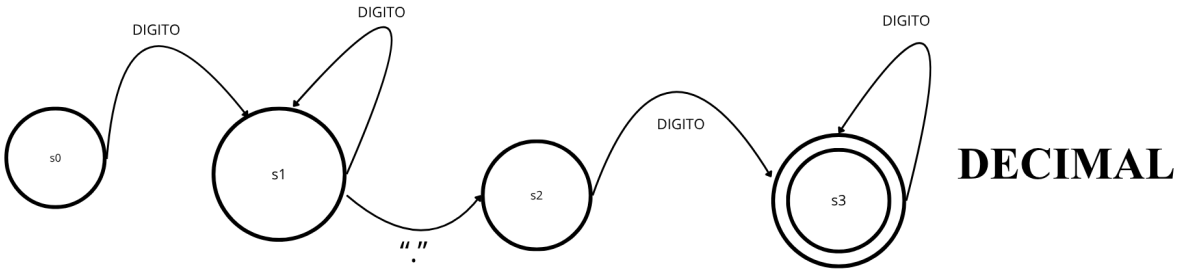
Específicos

Identificar tokens (enteros, decimales, operadores).
Implementar una gramática libre de contexto en Bison.
Manejar errores sintácticos y léxicos de manera controlada.

4. Desarrollo

Esquema del Automata para numeros ENTEROS Y DECIMALES





Notacion BNF

```
<entrada> ::= <expresion> EOL | EOL
<expresion>   ::= <expresion> "+" <termino>
                | <expresion> "-" <termino>
                | <termino>
<termino>     ::= <termino> "*" <factor>
                | <termino> "/" <factor>
                | <factor>
<factor>      ::= "(" <expresion> ")"
                | ENTERO
                | DECIMAL
```

Código flex

```
%{
 #include<stdio.h>
 #include<stdlib.h>
 #include "parser.tab.h"
%}
DIGITO [0-9]
ID [a-zA-Z][a-zA-Z0-9]*
%%
{DIGITO}+ {yylval.num = atoi(yytext); return ENTERO;}
{DIGITO}+"."{DIGITO}+ {yylval.num=atof(yytext); return DECIMAL;}
"+" {return SUMA;}
"-" {return RESTA;}
"*" {return MULTI;}
"/" {return DIVI;}
">=" {return MAI;}
"<=" {return MEI;}
">" {return MAQ;}
"<" {return MEQ;}
"==" {return EQ;}
"!=" {return DIF;}
\n {return EOL;}
"(" {return P_I;}
")" {return P_D;}
{ID} {return ID;}
[ \t\r] {}
. {printf("error léxico: %s\n", yytext);}
%%
int yywrap(void){return 1;}
```

Código Bison

```
%{
 #include<stdio.h>
 #include <stdlib.h>
 int yylex();
 void yyerror( const char *s);
%}
%union {
 float num;
 char *id;
 }
// tokens
%token <num> ENTERO DECIMAL
%token <id> ID
%token SUMA RESTA MULTI DIVI
%token P_I P_D EOL DIF
%token MAQ MEQ MAI MEI EQ
%type <num> expresion termino factor
%%
call:
 call input
 |
 ;
input:
 expresion EOL {printf("Resultado: %.2f\n",$1);}
 |EOL
 ;
expresion:
  expresion SUMA termino {$$= $1+$3;}
 |expresion RESTA termino {$$= $1-$3;}
 |termino   {$$=$1;}
 ;
termino:
 termino MULTI factor {$$=$1*$3;}
 |termino DIVI factor { if($3==0){printf("ERROR: division por 0\n");$$=0;}else{ $$=$1/$3;}}
 |factor       {$$=$1;}
 ;
factor:
 P_I expresion P_D {$$=$2;}
 |ENTERO  {$$=$1;}
 |DECIMAL {$$=$1;}
 ;
%%
void yyerror(const char*s){
 printf("Error sintantico: %s\n",s);
}
int main(){
 printf("Ingrese una expresión mateatica:\n");
 yyparse();
 return 0;
}
```

Pruebas

El analizador acepta operaciones aritmeticas, evalua que esten escritas de forma correcta y las resuelve tomando en cuenta la jerarquia de operaciones. Si la operacion esta bien escrita o es valida, el programa arrojara el resultado de la operacion; En caso contrario arrojara un mensaje de error "Syntax Error" como una calculadora.

para la compilacion del codigo se siguieron los siguientes pasos desde la terminal cmd de la computadora.
Se obtienen 3 archivos "lexer.l" para el analizador lexico "parser.y" para el analizador sintactico.
primero se determina la direccion en la que estan los archivos .l y .y para su compilacion; y desde el cmd se ejecutan los siguientes comandos.

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>flex lexer.l
```

- esta parte crea el archivo lex.yy.c que contiene un include "parser.tab.h" para vincularse con el otro analizador sintactico

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>bison -d parser.y
```

-esta parte crea los archivos parser.tab.y y parser.tab.h para su ejecucion

-como ultimo paso se genera el .exe de la ejecucion del programa con el comando:

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>gcc parser.tab.c lex.yy.c -o analizador.exe
```

```
se ingresan las siguientes entradas puestas en archivos de texto:

correctas:
(5.2+9)/2*10
(8+9)*5/2
5+3*2


incorrectas:
 5 + * 3
(10 - 2
4.5.6 + 1
```

Se ingresan las entradas correctas por medio de un archivo .txt desde la consola con el comando:

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>analizador.exe<entradas_correctas.txt
```

y se ontienen los siguientes resultados:

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>analizador.exe<entradas_correctas.txt
Ingrese una expresi%n mateatica:
Resultado: 71.00
Resultado: 42.50
Resultado: 11.00
```

por ultimo se ingresan las operaciones incorrectas con el mismo comando llamando a un archivo .text desde la consola

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>analizador.exe<entradas_incorrectas.txt
```

```
C:\Users\ado77\OneDrive\Escritorio\AyC_practicas>analizador.exe<entradas_incorrectas.txt
Ingrese una expresi%n mateatica:
Error sintantico: syntax error
```

5. Conclusiones

En conclusión, el proyecto permitió materializar la abstracción teórica de las Gramáticas de Backus-Naur (BNF). Se verificó que el analizador sintáctico es el eje rector que dota de semántica y orden operativo a las unidades léxicas; sin esta etapa de procesamiento, el flujo de información carecería de la estructura jerárquica necesaria para la resolución de expresiones. La práctica confirma que la robustez de un lenguaje depende directamente de la precisión con la que el parser interpreta y vincula los valores lógicos extraídos por el scanner.

Referencias Bibliográficas

Carranza Sahagún, D. U. (Coord.). (2024).

Compiladores: Fases de análisis. Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). Lenguajes formales y teoría de automátas. Editorial Alfaomega. Argentina.

Gutú, O. (2013). Primer curso en teoría de autómatas y lenguajes formales. Pearson Educación. México.

Hopcroft, J. E. Motwani, R. Ullman, J. D. (2007). Introducción a la teoría de automatas, lenguajes y computación. Pearson Educación. México.